

```

import heapq

# Goal State
goal = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 0]
]

# Manhattan Distance Heuristic
def heuristic(state):
    distance = 0
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                goal_x = (value - 1) // 3
                goal_y = (value - 1) % 3
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance

# Find blank tile
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            neighbors.append(new_state)
    return neighbors

# A* Search Algorithm
def a_star(start):
    visited = set()
    pq = []
    heapq.heappush(pq, (heuristic(start), 0, start, []))

```

```

while pq:
    f, g, state, path = heapq.heappop(pq)

    state_tuple = tuple(map(tuple, state))
    if state_tuple in visited:
        continue

    visited.add(state_tuple)
    path = path + [state]

    if state == goal:
        return path

    for neighbor in get_neighbors(state):
        if tuple(map(tuple, neighbor)) not in visited:
            heapq.heappush(
                pq,
                (g + 1 + heuristic(neighbor), g + 1, neighbor, path)
            )

return None

```

Initial State

```

start = [
    [1, 2, 3],
    [4, 0, 6],
    [7, 5, 8]
]

```

Solve Puzzle

```

solution = a_star(start)

```

if solution:

```

    print("Solution found in", len(solution) - 1, "moves:\n")

```

```

    for step in solution:

```

```

        for row in step:

```

```

            print(row)

```

```

        print()

```

else:

```

    print("No solution found.")

```

Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/h1/OneDrive/Desktop/AI/expp.py =====
Solution found in 2 moves:

[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]

>>>

expp.py - C:/Users/h1/OneDrive/Desktop/AI/expp.py (3.10.0)
File Edit Format Run Options Window Help
import heapq

Goal State
goal = [
 [1, 2, 3],
 [4, 5, 6],
 [7, 8, 0]
]

Manhattan Distance Heuristic
def heuristic(state):
 distance = 0
 for i in range(3):
 for j in range(3):
 value = state[i][j]
 if value != 0:
 goal_x = (value - 1) // 3
 goal_y = (value - 1) % 3
 distance += abs(i - goal_x) + abs(j - goal_y)
 return distance

Find blank tile
def find_blank(state):
 for i in range(3):
 for j in range(3):
 if state[i][j] == 0:
 return i, j

Generate neighboring states
def get_neighbors(state):
 neighbors = []
 x, y = find_blank(state)
 moves = [(-1,0), (1,0), (0,-1), (0,1)]

 for dx, dy in moves:
 nx, ny = x + dx, y + dy
 if 0 <= nx < 3 and 0 <= ny < 3:
 new_state = [row[:] for row in state]
 new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
 neighbors.append(new_state)
 return neighbors

A* Search Algorithm
def a_star(start):
 visited = set()
 pq = []
 heapq.heappush(pq, (heuristic(start), 0, start, []))

 while pq:

Ln: 19 Col: 0Ln: 14 Col: 20